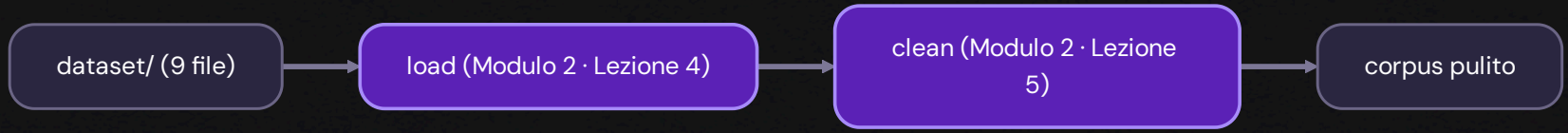
Lab Data Engineering

Modulo 2 · Lezione 6 · Lab



Dove siamo

Da Modulo 2 · Lezione 4 sappiamo **caricare** (`load()` → `Document`), da Modulo 2 · Lezione 5 **pulire** (`clean`) — entrambi nel toolkit `scaffolding/aikit/` . Ma finora: documenti scelti bene, uno alla volta. Oggi le due cose **insieme**, su un dataset intero — come promesso: *«sui PDF con tabelle e scansioni il parsing nativo va in crisi»* (Modulo 2 · Lezione 4), *«dati reali sporchi»* (Modulo 2 · Lezione 5).



Il corpus che esce oggi è la **materia prima** di tutto il resto del modulo.

> 4 ore, quasi tutte **vostre**: il docente apre ogni blocco, il codice lo scrivete voi.

Agenda

Le slide sono il filo: quando c'è da lavorare, una **slide di switch** vi manda al posto giusto (file, tempo) — finita l'attività, si torna qui. 📊 = slide (brief e debrief) · 🎤 = live coding del docente (il banner dice se replicate o se il codice è già nel file) · ⌨️ = scrivete voi (docente in affiancamento) · 🌐 = tutti al browser

	Blocco	Dove si lavora	Tempo
📊	Brief + metodo	slide	20′
🎤	Demo guida — il primo giro	in testa a `working/triage.py`	25′
⌨️	**Lab 1 — Triage** (debrief incluso)	`working/triage.py`	40′
🌐🎤	LlamaParse: registrazione + demo	`cloud.llamaindex.ai`	20′
⌨️🌐	**La caccia al tier** (playground)	`working/scheda_tier.md`	30′
☕	Pausa	—	15′
🎤	Live coding: il loader	`working/llamaparse.py`	15′
⌨️	**Lab 2 — Il routing**	`working/pulisci_corpus.py`	50′
📊	Debrief finale	`triage.md` · `scheda_tier.md` · `report.md`	25′



Il metodo del giorno

Un **ciclo**, ripetuto su ogni file del dataset.

Il ciclo

1. **carica** → `load(path)`
2. **guarda l'output** → quanti caratteri? che aspetto ha?
3. **diagnostica** → cosa manca, cosa è rotto, perché
4. **correggi** → il fix giusto per *quel* difetto

La regola del giorno

«**Esce del testo**» ≠ «esce il testo **giusto**». Un loader senza errori può comunque restituire un testo sbagliato.

Tra poco il docente fa il **primo giro completo** del ciclo su un documento — poi il metodo è vostro.



Il dataset: Lumen S.r.l.

I documenti interni della stessa Lumen di Modulo 2 · Lezione 4-5: policy, listino, verbali, contratti. Arrivano **come arrivano sempre**: da scanner, da export, da siti web.

- `guida_smartworking.pdf`
- `verbale_scansionato.pdf`
- `rassegna_stampa.pdf`
- `faq_prodotto.html`
- `note_interne.txt`
- `listino_prezzi.pdf`
- `fattura_scansionata.pdf`
- `report_trimestrale.pdf`
- `contratto_fornitura.docx`

La sfida

Più della metà vi darà problemi — e non tutti lo stesso problema. Quali, e perché, lo scoprite voi al Lab 1.

Setup (5') — copia-incolla, riga per riga

Terminale aperto nella cartella della lezione, poi:

```
cd scripts
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
```

Ogni comando si lancia da `scripts/`, si lavora in `working/`; `scaffolding/aikit/` si importa, non si modifica. Un comando fallisce? **Alzate la mano**. Chiave: **più tardi, in classe**.



Demo guida: il primo giro

🎤 **LIVE CODING — il docente lo ricostruisce, voi ce l'avete già · 25'**
`working/triage.py` (il "primo giro", già scritto in testa al file)

si torna alle slide
per la consegna del Lab 1

Il docente fa il ciclo **completo** su `faq_prodotto.html` — un file che si carica «senza errori». Lanciatelo anche voi: `python working/triage.py` stampa il giro anche coi TODO ancora da fare.

1 · carica — `load(path)`, nessuna eccezione. Tutto bene?
2 · guarda — 4.515 caratteri… che si aprono con titolo e cookie banner
3 · diagnostica — il contenuto c'è, annegato nel boilerplate web
4 · correggi — un primo fix mirato: via le righe del banner (il fix sistematico è il Lab 2)

> Questo ciclo è esattamente ciò che farete voi su **tutto** il dataset — file che si rompono inclusi.



1. Lab 1 — Il triage



Consegna Lab 1 — Triage

⌨ **TOCCA A VOI · 40'**
`working/triage.py` (stesso file della demo · soluzione in `solutions/` a fine lab)

si torna alle slide per il debrief

Un esito per **ogni** file del dataset — **OK · VUOTO · ERRORE** — più l'anteprima del testo.

TODO 1 · `carica_testo` — tutto il testo in una stringa; se utf-8 fallisce, rileggi in **latin-1**
TODO 2 · `triaga_file` — try/except sul load, poi classifica: `(esito, dettaglio, anteprima)`
TODO 3 · `triage_dataset` — tutto il dataset → la tabella

Fatto quando

`python working/triage.py` (dalla radice `scripts/`) stampa la **tabella completa** e scrive `triage.md`. Il main gira a stadi: ogni TODO si vede appena è fatto.

La colonna che conta

L'**anteprima**: è lì che vedrete cose strane. Non fidatevi degli esiti "OK" — **leggeteli**. Finito prima? Rileggete le anteprime dei file OK: **almeno due** nascondono comunque un problema.



Debrief Lab 1: i 4 modi di fallire

📊 AL RIENTRO Confrontiamo i triage: ogni file problematico fallisce in un modo **diverso** — e la differenza è tutto.

1 · Rumoroso

Il loader **solleva un'eccezione**: si vede subito. Fastidioso, ma onesto.

2 · Silenzioso

Nessun errore, ma **zero testo** estratto. Se non conti i caratteri, non te ne accorgi.

3 · Subdolo

Il testo c'è, ma è **spappolato**: la struttura (una tabella…) è morta. L'esito dice OK.

4 · Invisibile

Testo plausibile, ma **manca un pezzo** — e nessuno te lo dice. Il peggiore di tutti.

I primi due li becca **uno script**. Gli ultimi due li becca solo **chi guarda** — per questo la tabella ha l'anteprima. Quale file era cosa?



2. LlamaParse — il parser gestito



Il limite di pypdf: tabelle e scansioni

Dal triage: il listino **si carica senza errori**, ma guardate l'anteprima. Le celle della tabella escono una per riga, alla rinfusa.

Cosa esce da pypdf

```
Codice
Prodotto
Prezzo unitario
Sconto volume
LD-100
LumaDesk Basic 120×70
129,00
…
```

Quale prezzo è di quale prodotto? La struttura è **morta**.

Cosa vorremmo

Codice	Prodotto	Prezzo
LD-200	LumaDesk Pro	249,00

La tabella **ricostruita**, righe e colonne al loro posto.

E la scansione? **O caratteri**: lì serve l'OCR. Scrivere noi parser di layout + OCR = settimane. Oppure: si **delega**.



LlamaParse: cos’è, e i suoi tier

LlamaParse è un servizio gestito che fa il parsing "difficile" — layout, tabelle, scansioni (OCR) — e restituisce **markdown**. Non ha un solo motore: ha **4 tier**, e ognuno costa diverso. Free tier ≈ **10.000 crediti/mese**.

Tier	Crediti/pagina	Che cosa sa fare
`fast`	1	solo estrazione del testo (OCR incluso) — niente struttura
`cost_effective`	3	testo + **tabelle** e layout: il default ragionevole
`agentic`	10	i layout difficili che il 3-crediti sbaglia
`agentic_plus`	45	i casi disperati

Regola del giorno #2: **il tier giusto è il più basso che risolve il difetto di QUEL file** — non il più potente. E si decide **provando**, non a catalogo: è la caccia al tier.

> I vincoli restano: **latenza** (job asincrono, decine di secondi) e documenti che **viaggiano nel cloud** — ne riparliamo al debrief.



Alla webapp: registrazione + playground

🌐 **AL BROWSER — registrazione VOSTRA, demo del DOCENTE · 20'**

`cloud.llamaindex.ai` → poi la chiave nel `.env`

si torna alle slide
per la caccia al tier

⌨️ Voi (~10')

Registratevi (free tier) e create la **API key** → aprite `scripts/.env` con l'editor e incollatela dopo l'uguale: `LLAMA_CLOUD_API_KEY=llx-…` Chi si blocca **alza la mano**: il docente vi passa la chiave di riserva, registrerete dopo.

🎤 Il docente (~10')

A schermo, il **giro completo** nel playground su `listino_prezzi.pdf` : upload, scelta del **tier**, output markdown, e dove si leggono i **crediti consumati** in dashboard. È il giro che tra poco farete voi, sui vostri file.

> Subito dopo tocca a voi: la **caccia al tier** sui file del vostro triage.



La caccia al tier



TOCCA A VOI, NEL PLAYGROUND · 30'

`working/scheda_tier.md` (la scheda da compilare)

poi ☕ 15',
quindi slide → il loader

I **5 file candidati** del vostro triage: le 3 scansioni, il listino, il contratto. Per ciascuno:

1. caricatelo nel playground e provatelo su **almeno 2 tier**
2. guardate l'output: il **difetto del triage è risolto**? (la tabella è tornata tabella? le colonne sono in ordine?)
3. annotate i **crediti consumati** (dashboard) e scegliete: qual è il tier **più basso che basta**?

Fatto quando

`scheda_tier.md` compilata: tier provati, crediti, difetto risolto?, **tier scelto e perché**. Diventa il vostro `ROUTING` al Lab 2.

Budget

Una caccia completa costa **~30–40 crediti** su 10.000: sperimentate pure. I job sono **asincroni**: lanciatene uno e intanto leggete l'altro. Non chiudete tutti e 5? Va bene: **meglio 3 file indagati bene** che 5 a caso.



Dalla GUI al codice: è solo un'API

Quello che avete appena fatto a mano nella webapp, ora lo automatizziamo. Niente SDK: **tre chiamate** `httpx` — e il **tier** è solo un campo della richiesta.



Dietro la STESSA interfaccia

`load_llamaparse(path, tier=…) → list[Document]` — il resto della pipeline **non sa** chi ha fatto il parsing. Si innesta, non sostituisce.

Da sapere

Il job è **asincrono**: si aspetta (`sleep` , non panico). L'SDK storico è **deprecato** e si porta dietro llama-index; il nuovo `llama-cloud` è pulito — in produzione va benissimo, qui vogliamo vedere il job da vicino.



Live coding: il loader

🎤 **LIVE CODING — si guarda e si replica · 15'**
`working/llamaparse.py` (soluzione in `solutions/`)

si torna alle slide
per la consegna del Lab 2

Il docente scrive dal vivo `load_llamaparse(path, tier=…)` — il **TODO 4**, coi passi anche nel docstring. Voi **replicate nel vostro `working/`** mentre lo costruisce: a fine live il loader ce l'avete anche voi, con il tier come parametro.

Fatto quando

Il loader è **replicato per intero** nel vostro `working/llamaparse.py`. La prova vera è al Lab 2: il report dirà tier e **crediti** dei file instradati.

Da notare, mentre replicate

Il **tier** è solo un campo JSON della seconda chiamata; la validazione (`ValueError` sul tier, `SystemExit` senza chiave) avviene **prima** di toccare la rete.



3. Lab 2 — Il corpus pulito



Consegna Lab 2 — Il routing

⌨ **TOCCA A VOI · 50'**
`working/pulisci_corpus.py` (soluzione in `solutions/` — a fine lab)

si torna alle slide per il debrief

La caccia al tier diventa **pipeline riproducibile**: chiunque rilancia lo script e ottiene lo stesso corpus. Indipendente dal Lab 1, il cleaning di Modulo 2 · Lezione 5 è **fornito** — oggi la decisione è un'altra:

TODO 5 · `ROUTING` — per ognuno dei 5 file candidati: `"nativo"` oppure il tier scelto nella **vostra** scheda. Il tier giusto è il più **basso** che risolve il difetto — le scansioni non si aggiustano gratis, ma nemmeno tutto merita l'`agentic`.

Fatto quando

`python working/pulisci_corpus.py` (da `scripts/`) → `corpus_pulito/` + `report.md` con loader, **tier e crediti per file**, senza note di degrado. Senza chiave gira offline (soli nativi): comodo per sistemare il `ROUTING` senza spendere.

Stretch

Provate l'`agentic` su un file già a posto col `cost_effective`: cambia qualcosa? Documentatelo — è un risultato anche il «no».



Debrief: locale o gestito?

📊 AL RIENTRO La risposta è nel vostro `report.md`.

Nativo / locale

gratis · offline · i documenti **restano da voi** · veloce

…ma sul difficile (tabelle, scansioni) **perde**

Gestito (LlamaParse)

robusto sul difficile · markdown pronto

…ma a pagamento (crediti — Modulo 2 · Lezione 2!) · più lento (job asincrono) · i documenti **viaggiano nel cloud**

Quando nessun parser basta

Il PDF troncato non lo salva **nessun** parser: si diagnostica e si chiede una copia sana.

Si sceglie **per file e per tier, coi numeri** — nel nostro corpus: 5 file su 9 gestiti, e non tutti allo stesso tier (il verbale basta `fast`). Portate `triage.md`, `scheda_tier.md` e `report.md`.



Cosa portate a casa

Un corpus, un **metodo** (carica → guarda → diagnostica → correggi) e un toolkit più ricco.

I deliverable

`triage.md` — la diagnosi
`scheda_tier.md` — le prove, coi crediti misurati
`pulisci_corpus.py` + `ROUTING` — la cura, riproducibile
`corpus_pulito/` + `report.md` — l'esito, tier e crediti per file
`llamaparse.py` — il loader gestito, **vostro**

Prossimi passi

Il corpus pulito è la **materia prima** di tutto quello che costruiremo da qui a fine modulo. Dalla prossima lezione iniziamo a usarlo.

> **load** (Modulo 2 · Lezione 4) e **clean** (Modulo 2 · Lezione 5) usati insieme su dati veri. Il loader gestito resta vostro, pronto da innestare quando servirà.



Grazie

La prossima lezione: Modulo 2 · Lezione 7 — Embeddings

